

```

# =====
# PLANT DISEASE DETECTION - MobileNetV2 (KaggleHub Version)
# =====

# STEP 1: Install & Imports
!pip install kagglehub -q

import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import kagglehub

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from google.colab import files

print("✅ TF Version:", tf.__version__)
print("✅ GPU:", tf.config.list_physical_devices('GPU'))

# =====
# STEP 2: Download Dataset using KaggleHub
# =====
print("\n📄 Downloading PlantVillage Dataset...")

path = kagglehub.dataset_download("adilmubashirchaudhry/plant-village-dataset")
print("✅ Dataset Path:", path)

# =====
# STEP 3: Auto-Detect Train & Val Folders
# =====
print("\n📁 Scanning dataset structure...")

# Show full structure
for root, dirs, files_list in os.walk(path):
    level = root.replace(path, '').count(os.sep)
    indent = ' ' * level
    print(f"{indent} 📁 {os.path.basename(root)}")
    if level < 2:
        for f in files_list[:3]:
            print(f"{indent} 📄 {f}")

# Auto-find train & val directories
train_dir = None
val_dir = None

for root, dirs, _ in os.walk(path):
    for d in dirs:
        full = os.path.join(root, d)
        if d.lower() in ['train', 'training']:
            train_dir = full
        if d.lower() in ['val', 'valid', 'validation', 'test']:
            val_dir = full

# If no train/val split found – use full dataset with split
if not train_dir:
    print("\n⚠️ No train/val folders found – using full dataset with 80/20 split")
    train_dir = path
    val_dir = None

print(f"\n✅ Train Dir : {train_dir}")
print(f"✅ Val Dir : {val_dir if val_dir else 'Auto-split from train'}")

# Get class names
classes = sorted([d for d in os.listdir(train_dir)
                  if os.path.isdir(os.path.join(train_dir, d))])
num_classes = len(classes)
print(f"✅ Classes : {num_classes}")
print(f" {classes[:5]} ...")

# Save class names
with open("class_names.txt", "w") as f:
    for c in classes:

```

```

        f.write(c + "\n")
    print("✅ class_names.txt saved!")

# =====
# STEP 4: Data Generators
# =====
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

if val_dir:
    # Separate train & val folders exist
    train_gen = ImageDataGenerator(
        rescale = 1./255,
        rotation_range = 20,
        zoom_range = 0.2,
        horizontal_flip = True,
        width_shift_range = 0.1,
        height_shift_range = 0.1,
    )
    val_gen = ImageDataGenerator(rescale=1./255)

    train_data = train_gen.flow_from_directory(
        train_dir,
        target_size = IMG_SIZE,
        batch_size = BATCH_SIZE,
        class_mode = 'categorical',
        shuffle = True
    )
    val_data = val_gen.flow_from_directory(
        val_dir,
        target_size = IMG_SIZE,
        batch_size = BATCH_SIZE,
        class_mode = 'categorical',
        shuffle = False
    )

else:
    # No split - use validation_split
    train_gen = ImageDataGenerator(
        rescale = 1./255,
        rotation_range = 20,
        zoom_range = 0.2,
        horizontal_flip = True,
        width_shift_range = 0.1,
        height_shift_range = 0.1,
        validation_split = 0.2
    )

    train_data = train_gen.flow_from_directory(
        train_dir,
        target_size = IMG_SIZE,
        batch_size = BATCH_SIZE,
        class_mode = 'categorical',
        subset = 'training',
        shuffle = True
    )
    val_data = train_gen.flow_from_directory(
        train_dir,
        target_size = IMG_SIZE,
        batch_size = BATCH_SIZE,
        class_mode = 'categorical',
        subset = 'validation',
        shuffle = False
    )

print(f"\n✅ Train : {train_data.samples} images")
print(f"✅ Val : {val_data.samples} images")

# =====
# STEP 5: Build MobileNetV2 Model
# =====
print("\n🚧 Building MobileNetV2 Model...")

base_model = MobileNetV2(
    input_shape = (224, 224, 3),
    include_top = False,
    weights = 'imagenet'
)

```

```

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(num_classes, activation='softmax')
])

model.summary()

# =====
# STEP 6: Phase 1 – Train Top Layers
# =====
print("\n" + "="*60)
print("🚀 PHASE 1 – Training Top Layers")
print("="*60)

model.compile(
    optimizer = tf.keras.optimizers.Adam(learning_rate=0.001),
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=4,
                  restore_best_weights=True, verbose=1),
    ModelCheckpoint('best_disease_model.h5', monitor='val_accuracy',
                   save_best_only=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                     patience=2, min_lr=1e-6, verbose=1)
]

history1 = model.fit(
    train_data,
    validation_data = val_data,
    epochs         = 10,
    callbacks      = callbacks,
    verbose        = 1
)

# =====
# STEP 7: Phase 2 – Fine Tuning
# =====
print("\n" + "="*60)
print("🔧 PHASE 2 – Fine Tuning Last 30 Layers")
print("="*60)

base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False

model.compile(
    optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)

history2 = model.fit(
    train_data,
    validation_data = val_data,
    epochs         = 10,
    callbacks      = callbacks,
    verbose        = 1
)

# =====
# STEP 8: Training Curves
# =====
acc      = history1.history['accuracy'] + history2.history['accuracy']
val_acc  = history1.history['val_accuracy'] + history2.history['val_accuracy']
loss     = history1.history['loss'] + history2.history['loss']
val_loss = history1.history['val_loss'] + history2.history['val_loss']

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

```

```

fig.suptitle('MobileNetV2 Training History', fontsize=14, fontweight='bold')

axes[0].plot(acc, label='Train Acc', color='steelblue', linewidth=2)
axes[0].plot(val_acc, label='Val Acc', color='coral', linewidth=2)
axes[0].axvline(x=len(history1.history['accuracy'])-1,
                color='gray', linestyle='--', label='Fine-tune start')
axes[0].set_title('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(loss, label='Train Loss', color='steelblue', linewidth=2)
axes[1].plot(val_loss, label='Val Loss', color='coral', linewidth=2)
axes[1].axvline(x=len(history1.history['loss'])-1,
                color='gray', linestyle='--', label='Fine-tune start')
axes[1].set_title('Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()

# =====
# STEP 9: Final Evaluation
# =====
print("\n" + "="*60)
print(" 🏁 FINAL EVALUATION")
print("="*60)

val_loss_final, acc_final = model.evaluate(val_data, verbose=1)
print(f"\n 🏁 Final Validation Accuracy : {acc_final*100:.2f}%")
print(f" 🏁 Final Validation Loss : {val_loss_final:.4f}")

# =====
# STEP 10: Download All Files
# =====
print("\n 📁 Downloading files...")

from google.colab import files as colab_files

for fname in ["best_disease_model.h5",
              "class_names.txt",
              "training_curves.png"]:
    try:
        colab_files.download(fname)
        print(f" ✅ {fname}")
    except Exception as e:
        print(f" ⚠️ {fname} - {e}")

print("\n" + "="*60)
print(" 🏁 TRAINING COMPLETE!")
print(f" ✅ best_disease_model.h5 → Website ூ use చేయ")
print(f" ✅ class_names.txt → Disease names")
print("="*60)

```

✓ TF Version: 2.19.0

✓ GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

⬇ Downloading PlantVillage Dataset...

Downloading to /root/.cache/kagglehub/datasets/adilmubashirchaudhry/plant-village-dataset/1.archive...
100%|██████████| 989M/989M [01:10<00:00, 14.7MB/s]Extracting files...

✓ Dataset Path: /root/.cache/kagglehub/datasets/adilmubashirchaudhry/plant-village-dataset/versions/1

📁 Scanning dataset structure...

📁 1/

📁 PlantVillageDataset/

📁 PlantVillage/

📁 PlantVillage/

📁 Pepper__bell__healthy/

📁 Potato__Early_blight/

📁 Tomato_healthy/

📁 Potato__Late_blight/

📁 Tomato__Target_Spot/

📁 Tomato_Septoria_leaf_spot/

📁 Tomato_Bacterial_spot/

📁 Tomato__Tomato_mosaic_virus/

📁 Tomato__Tomato_YellowLeaf__Curl_Virus/

📁 Pepper__bell__Bacterial_spot/

📁 Tomato_Leaf_Mold/

📁 Potato__healthy/

📁 Tomato_Early_blight/

📁 Tomato_Spider_mites_Two_spotted_spider_mite/

📁 Tomato_Late_blight/

📁 Pepper__bell__healthy/

📁 Potato__Early_blight/

📁 Tomato_healthy/

📁 Potato__Late_blight/

📁 Tomato__Target_Spot/

📁 Tomato_Septoria_leaf_spot/

📁 Tomato_Bacterial_spot/

📁 Tomato__Tomato_mosaic_virus/

📁 Tomato__Tomato_YellowLeaf__Curl_Virus/

📁 Pepper__bell__Bacterial_spot/

📁 Tomato_Leaf_Mold/

📁 Potato__healthy/

📁 Tomato_Early_blight/

📁 Tomato_Spider_mites_Two_spotted_spider_mite/

📁 Tomato_Late_blight/

📁 train_val_test/

📁 val/

📁 Pepper__bell__healthy/

📁 Potato__Early_blight/

📁 Tomato_healthy/

📁 Potato__Late_blight/

📁 Tomato__Target_Spot/

📁 Tomato_Septoria_leaf_spot/

📁 Tomato_Bacterial_spot/

📁 Tomato__Tomato_mosaic_virus/

📁 Tomato__Tomato_YellowLeaf__Curl_Virus/

📁 Pepper__bell__Bacterial_spot/